

# CS302: AUTOMATA THEORY AND FORMAL LANGUAGES

NORTHWEST SAMAR STATE UNIVERSITY

*Actualizing vision | Harnessing potentials | Improving lives*



# MODULE 1: INTRODUCTION TO AUTOMATA THEORY & FORMAL LANGUAGES

NORTHWEST SAMAR STATE UNIVERSITY

*Actualizing vision | Harnessing potentials | Improving lives*



## Lesson Objectives

- Define automata theory and explain its significance in computer science
- Identify rea-world applications of automata theory
- Explain the basic concepts: alphabet, string, language, and grammar.
- Differentiate the levels of the Chomsky Hierarchy with examples.

**When you use a particular machine, how does the machine know what you mean?**

## **This is exactly what Automata Theory studies:**

- ✓ How machines process inputs, follow rules, and decide what to accept or reject.

# What is Automata?

- ✓ (singular: Automaton) are abstract mathematical models of machines that process inputs and produce outputs according to a set of rules.
- ✓ Example: a vending machine:
  - ✓ Input = coins
  - ✓ State = “waiting for payment”, “ready to dispense”
  - ✓ Rule = if the amount is inserted, it moves to the “ready” state and gives you the item

## What is Automata?..cont.

- ✓ Comes from the Greek word **automatos** which means “**self-moving**” or “**acting on its own**”
- ✓ **Auto = self**
- ✓ **Matos = moving / acting**



# **Automata Theory or Theory of Computation**

- ✓ A branch of computer science that deals with the study of abstract machines and the problems they can solve.
- ✓ It also studies the formal languages that these automata can recognize.



# Importance of Automata Theory

- ✓ Forms the foundation of **compiler design** (lexical analyzers, parsers)
- ✓ Used in **pattern matching** (search engines, text editors, spam filters)
- ✓ Helps define the **limits of computation** (what computers can and cannot solve)
- ✓ Supports fields like **Artificial Intelligence, NLP, and networking**

# Automata Theory or Theory of Computation

- ✓ Different names:
  - Theory of Automata
  - Theory of Computer Science
  - Theory of Computation
  - Computer Theory

# **Applications of Automata Theory**

# Compiler Design

- ✓ **Where used:** In programming languages.
- ✓ **How:** Automata (specifically finite automata and pushdown automata) are used to design lexical analyzers and parsers.

# Text Search & Pattern Matching

- ✓ **Where used:** Search engines, word processors, spam filters.
- ✓ **How:** Automata check if a string matches a certain pattern.

# Networking Protocols

- ✓ **Where used:** Internet communication (TCP/IP, HTTP).
- ✓ **How:** Protocols are modeled as finite state machines (FSMs).

# Artificial Intelligence & Robotics

- ✓ **Where used:** Game AI, robots, decision-making systems
- ✓ **How:** Automata model decision-making and behaviors.
- ✓ Example: A simple robot vacuum cleaner:
  - State 1 = cleaning
  - State 2 = returning to charging dock
  - Input = battery low → move to State 2.



# Natural Language Processing (NLP)

- ✓ **Where used:** Chatbots, voice assistants (Siri, Alexa), grammar checkers.
- ✓ **How:** Automata model context-free grammars to analyze sentence structures.
- ✓ Example: Grammarly checks if a sentence is grammatically correct.

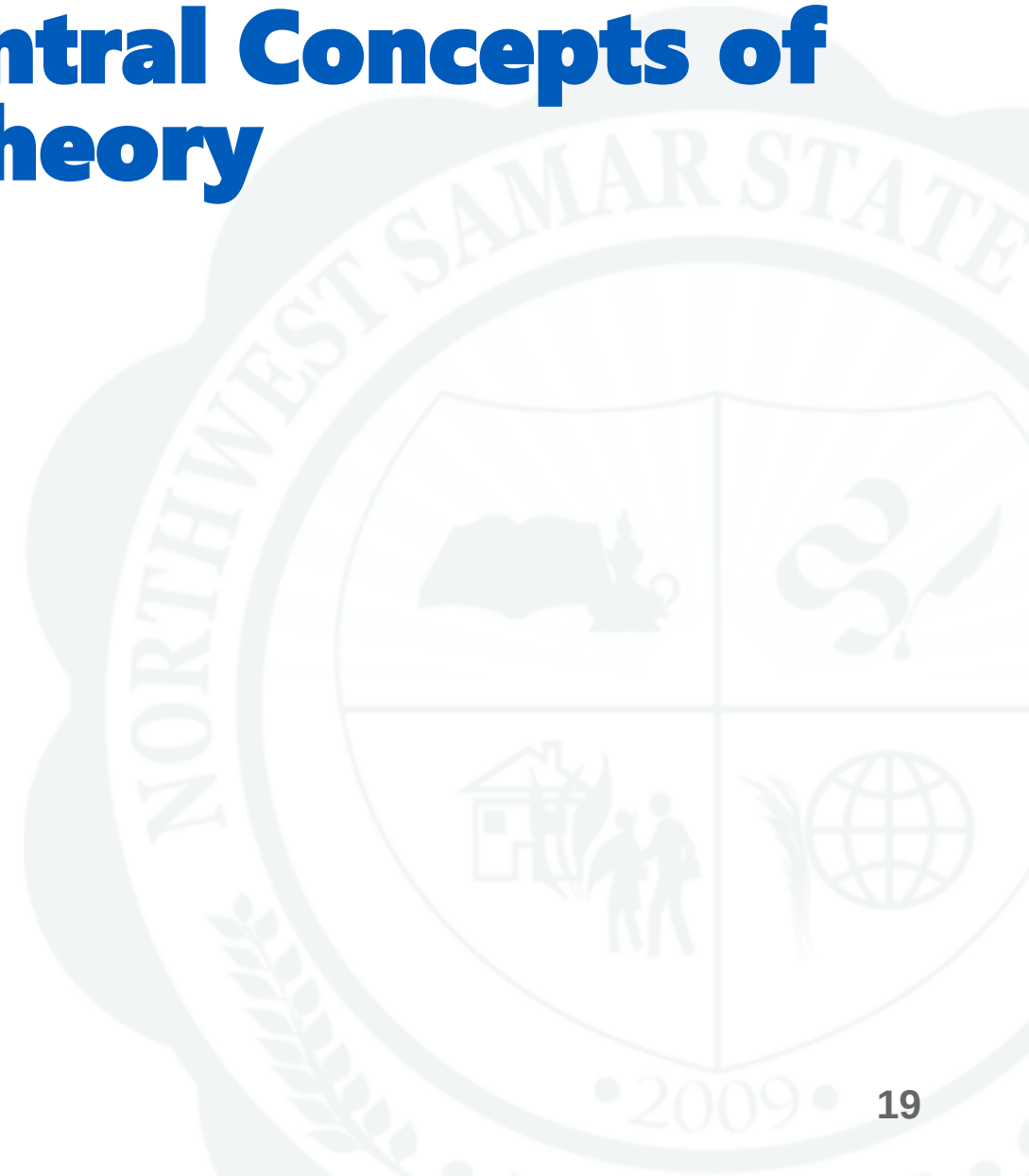
# Control Systems & Embedded Systems

- ✓ **Where used:** Elevators, traffic lights, washing machines.
- ✓ **How:** These systems are modeled as FSMs with states (idle, active, finished).
- ✓ Example: Washing machine cycle:
- ✓ Wash → Rinse → Spin → Stop.

# **Basic Fundamentals/Central Concepts of Automata Theory**

# **Basic Fundamentals/Central Concepts of Automata Theory**

1. Symbol
2. Alphabet
3. String or Word
4. Language
5. Grammar



# Symbol

- ✓ A **symbol** is an abstract entity
- ✓ indivisible objects or entity that cannot be defined
- ✓ **Example:**
  - Letters:** A to Z (upper case) or a to z (lower case)
  - Digits:** 0 to 9
  - Special Characters**

# Alphabet

An **alphabet** is a non-empty finite set of symbols. It is denoted by the symbol  $\Sigma$  (sigma)

## Examples:

$\Sigma = \{a, b, c\}$  is an alphabet which consisting of letters .

$\Sigma = \{0, 1\}$  is an alphabet which consisting of digits .

$\Sigma = \{a, 0\}$  is an alphabet which consisting of a letter 'a' and digit 0 .

Activate Windows  
Go to PC settings to activate Windows



# String or Word

A **string** or **word** is defined as finite sequence of symbols over an alphabet ( $\Sigma$ ).

## Examples:

1. Alphabet  $\Sigma = \{a, b\}$

Strings:

$w = \{a, b, aa, ab, ba, bb, aaa, aab, aba, baa, bbb, \dots\}$

2.  $\Sigma = \{0, 1\}$

$w = \{0, 1, 00, 01, 10, 11, 000, 001, 010, 100, 111, \dots\}$

# Language

- ✓ A **language (L)** set of strings formed from an alphabet that follow certain rules.
- ✓ Example:
  - **$L = \{ \text{all strings over } \{0,1\} \text{ that end with } 0 \}$**   
 **$= \{0, 10, 110, 1010, \dots\}$**
  - **$L = \{ a^n b^n \mid n \geq 1 \}$**   
 **$= \{ ab, aabb, aaabbb, \dots \}$**



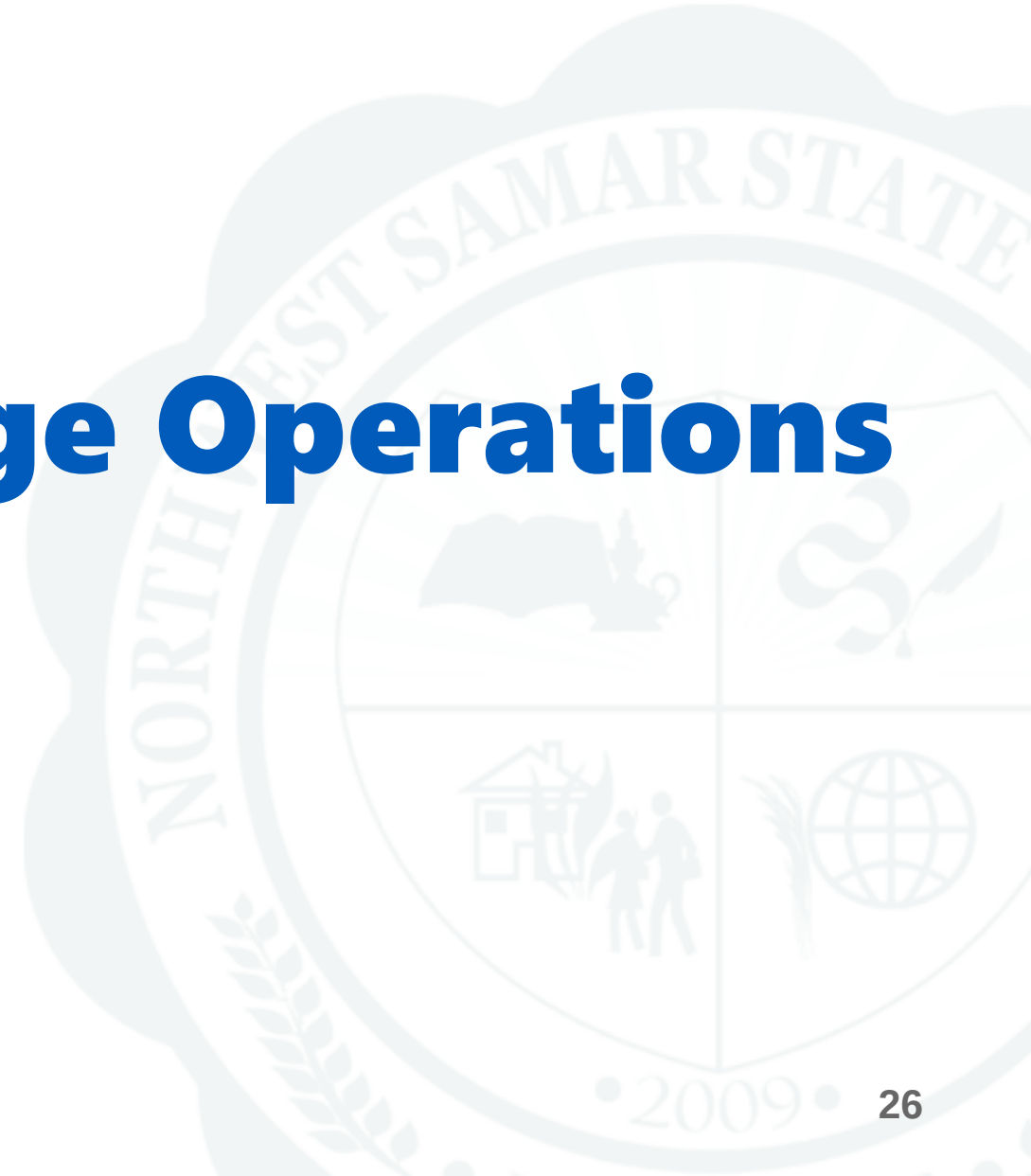
# Grammar

- ✓ A set of rules (productions) that describe how strings in a language can be generated.
- ✓ Example:
  - For the language  $L = \{ a^n b^n \mid n \geq 1 \}$ 
    - Rule 1:  $S \rightarrow aSb$
    - Rule 2:  $S \rightarrow ab$

# Summary:

- ✓ **Symbol:** entity that cannot be defined
- ✓ **Alphabet ( $\Sigma$ ):** Set of symbols
- ✓ **String ( $w$ ):** Sequence of symbols
- ✓ **Language ( $L$ ):** Set of valid strings
- ✓ **Grammar:** Rules to generate strings

# String and Language Operations



# String Operations



# String Operations

## ✓ Concatenation

- Joining two strings together.

The **concatenation** of two strings is the string formed by writing the first string followed by the second string with no space.

# String Operations

## i) Concatenation of strings:

### Examples:

Ex-1:

$w_1 = \text{DOG}, w_2 = \text{HOUSE}$

$w_1 w_2 = \text{DOGHOUSE}$

Ex-2:

$w_1 = x_1 x_2 \dots x_n, w_2 = y_1 y_2 \dots y_n$

$w_1 w_2 = x_1 x_2 \dots x_n y_1 y_2 \dots y_n$

# String Operations

## ✓ Length of a String

The **length** of a string **w**, denoted by **|w|** is the number of symbols composing the string.

### Examples:

Ex-1:

$w_1 = \text{abcd}$

$|w_1| = 4$



# String Operations

## ✓ Empty String

The **empty** string is denoted by  $\lambda$  (lambda) or  $\epsilon$  (epsilon) is the string consisting of **zero symbols** (i.e., no symbols).

i.e.,  $|\lambda|$  or  $|\epsilon| = 0$

### Note:

$\epsilon w = w \epsilon = w$  for any string ' $w$ '

# String Operations

- ✓ **Reverse of a String (Reversal)**
  - Writing the string in reverse order.

Let a string  $w = a_1 a_2 \dots a_n$  then the reverse of string  $w$  is denoted by  $w^R$  where  $R$  is reverse.

Therefore  $w^R = a_n a_{n-1} \dots a_2 a_1$

**Example:**

$w = abcd$

$w^R = dcba$

# String Operations

## ✓ Substring

- A sequence of consecutive symbols from the string

**Example:**

$w = xyz$  then  $xy, yz$  etc., are the substrings of  $w$ .

# String Operations

**Prefix:** A **prefix** of a string is any number of **leading symbols (starting)** of that string.

**Suffix:** A **suffix** of a string is any number of **trailing symbols (last)** of that string.

**Proper prefix or proper suffix:** A **prefix** or **suffix** of a string **other than the string itself** is called a **proper prefix** or a **proper suffix**.



# String Operations

## Examples:

Let  $w = xyz$

Prefixes:  $\epsilon, x, xy, xyz$

Suffixes:  $\epsilon, z, yz, xyz$

Proper prefixes:  $\epsilon, x, xy$

Proper suffixes:  $\epsilon, z, yz$

$w = 123$

$P = \epsilon, 1, 12, 123$

$S = \epsilon, 3, 23, 123$

$PP = \epsilon, 1, 12$

$PS = \epsilon, 3, 23$

# String Operations

## ✓ Sets of Strings

- $\Sigma^*$  (sigma closure) is the set of strings obtained by concatenating 0 (zero) or more symbols from  $\Sigma$ .
- $\Sigma^+$  (sigma positive closure) is the set of strings obtained by concatenating more symbols from  $\Sigma$  without 0 (zero).

Note:  $\Sigma$  is finite but  $\Sigma^*$  and  $\Sigma^+$  are infinite.

# String Operations

Example:

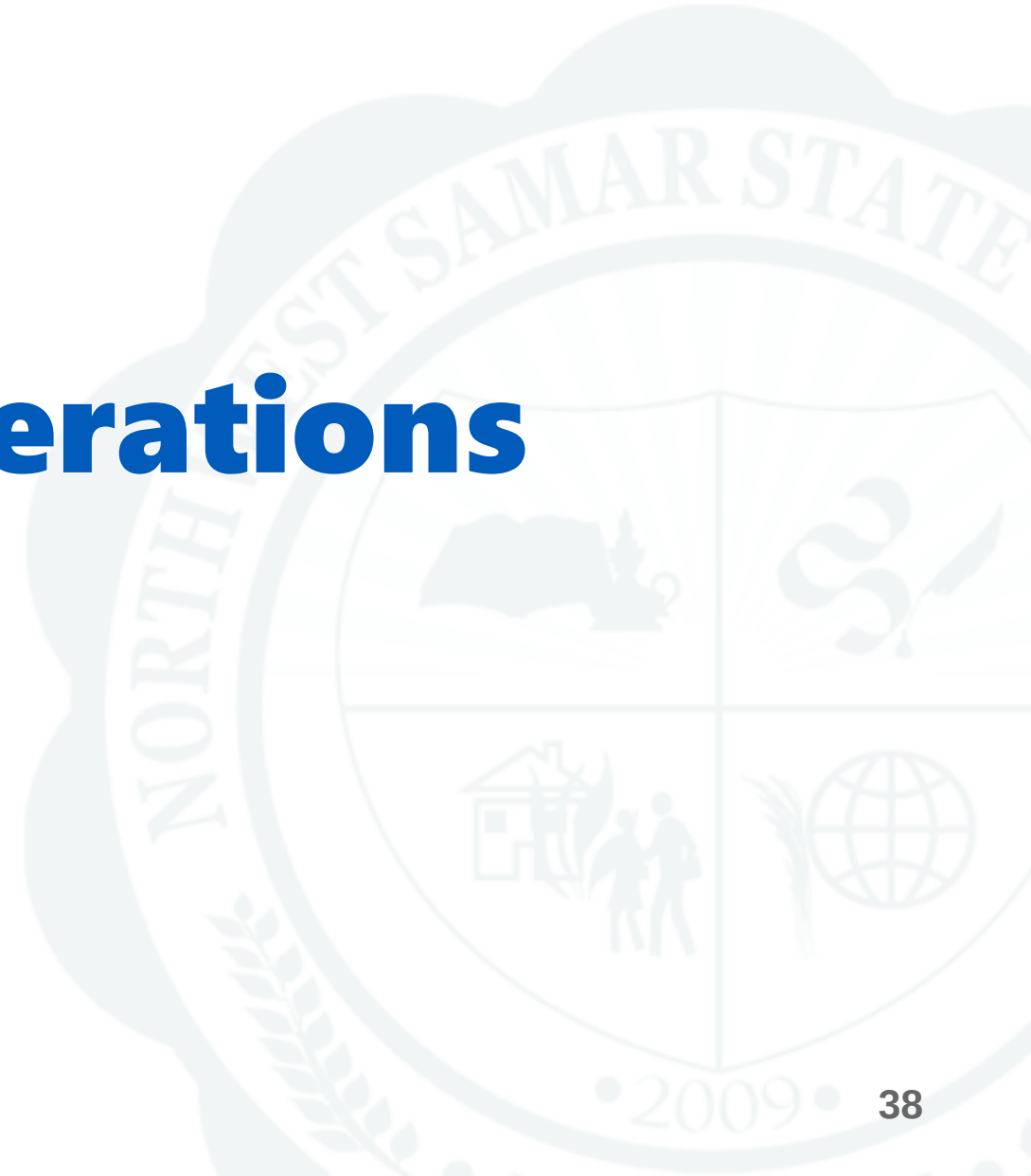
$$\Sigma = \{a, b\}$$

$$\Sigma^* = \{\epsilon, a, b, aa, ab, ba, bb, aaa, aab, aba, bab, bbb, \dots\}$$

$$\Sigma^+ = \{a, b, aa, ab, ba, bb, aaa, aab, aba, bab, bbb, \dots\}$$



# Language Operations



# Language Operations

## ✓ Compliment of a Language

The **compliment** of a language **L** is denoted by  $\bar{L}$  or  $L'$

and is defined as

$$L' = \Sigma^* - L$$

- ✓ All strings over  $\Sigma$  that are not in L

# Language Operations

## ✓ Compliment of a Language

- Example:
- $\Sigma = \{ 0 , 1 \}$  ,  $L = \{ 0, 11 \}$
- $L' = \{ \varepsilon, 1, 00, 01, 10, 111, \dots \}$

# Language Operations

## ✓ Reverse of a Language

The **reverse** of a language **L** is denoted by **L<sup>R</sup>** and is defined as

$$L^R = \{w^R \mid w \in L\} \text{ for string } w$$

**Example:**

$$L = \{xy \mid x, y \in w\}$$

$$L^R = \{yx \mid x, y \in w\}$$

# Language Operations

## ✓ Concatenation

- All strings formed by joining a string from L1 with the string from L2.
- Example:
  - $L1 = \{ a, b \}$  ,  $L2 = \{ c, d \}$
  - $L1L2 = \{ ac, ad, bc, bd \}$

# Language Operations

## ✓ Kleene Star $(L)^*$

- Zero or more concatenations of strings from  $L$ .
- Example:
  - $L = \{ a \}$
  - $L^* = \{ \epsilon, a, aa, aaa, \dots \}$
- $L^*$  always includes the empty string

# Language Operations

## ✓ Union ( $L1 \cup L2$ )

- All strings that belong to  $L1$ , or  $L2$ , or both.
- Example:
  - $L = \{ a, b \}$ ,  $L2 = \{ b, c \}$
  - $L1 \cup L2 = \{ a, b, c \}$



# Language Operations

## ✓ Intersection ( $L1 \cap L2$ )

- Strings that belong to both  $L1$  and  $L2$
- Example:
  - $L = \{ ab, bc \}, L2 = \{ bc, cd \}$
  - $L1 \cap L2 = \{ bc \}$

# Language Operations

## ✓ Difference ( $L1 - L2$ )

- Strings that belong to both  $L1$  but not in  $L2$
- Example:
  - $L = \{ a, b, c \}, L2 = \{ b, c \}$
  - $L1 - L2 = \{ a \}$

# Chomsky Hierarchy



# Chomsky Hierarchy of Formal Languages

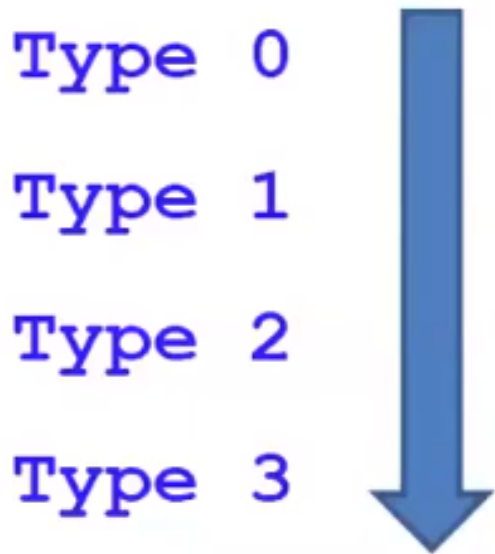
- ✓ The Chomsky Hierarchy (proposed by **Noam Chomsky** in 1956)
- ✓ A classification of **formal grammars** (and the languages they generate) into **four types**, based on their **generative power**.
- ✓ It shows the relationship between types of languages and the machines (automata) that recognize them.

# Chomsky Hierarchy of Formal Languages

| Type | Languages                             | Grammars                                           | Automata                      |
|------|---------------------------------------|----------------------------------------------------|-------------------------------|
| 0    | Recursively Enumerable Language (REL) | Phrase-structured/ Semi-true/Unrestricted grammars | Turing Machine (TM)           |
| 1    | Context-Sensitive Language (CSL)      | Context-Sensitive Grammars (CSG)                   | Linear-Bounded Automata (LBA) |
| 2    | Context-Free Language (CFL)           | Context-Free Grammars (CFG)                        | Pushdown Automata (PDA)       |
| 3    | Regular Language (RL)                 | Regular, Right-linear, Left-linear grammar         | Finite Automata (FA)          |

# Chomsky Hierarchy of Formal Languages

Computational power in decreasing order:



Type-3 < Type-2 < Type-1 < Type-0

# Chomsky Hierarchy of Formal Languages

